

Treinamento em Nextflow

Índice

1	Treinamento Nextflow	5
1.1	Visão geral do curso	5
2	Ambiente de trabalho	6
2.1	Ambiente Local	6
2.2	Ambiente Remoto	6
3	Objetivos	7
3.1	O que este curso não cobre	7
4	Pré-requisitos:	8
5	Agenda (5h)	9
I	Hello Nextflow	10
6	Hello Nextflow	11
6.1	O que é Nextflow	11
6.2	O ecossistema nf-core	11
6.3	Por que Nextflow	11
6.4	Defina o diretório de trabalho	12
7	Como um pipeline em Nextflow é estruturado?	13
7.1	Estrutura de um pipeline	13
7.2	Examinando um script nextflow	14
7.2.1	Componentes do hello-world.nf	15
7.2.2	Execute o workflow	15
7.2.3	Encontre a saída e os logs de execução	15
7.3	Aprimorando o Hello world	16
7.3.1	Publique outputs	16
7.3.2	Uso básico	16
7.4	Depois	16
7.5	Antes	17
7.6	Depois	17
7.7	Antes	17

7.8	Usando inputs variáveis	18
7.9	Depois	18
7.10	Antes	19
7.11	Depois	19
7.12	Antes	19
7.13	Depois	19
7.14	Antes	20
8	Gerenciando execução do workflow	21
8.0.1	Relance um fluxo de trabalho com <code>-resume</code>	21
8.0.2	Inspecione o log ou limpe execuções passadas	21
9	Componentes de um Workflow	23
9.1	Channels	23
9.1.1	Forneça entradas variáveis através de um canal explicitamente	23
9.2	Workflows	25
9.3	Adicionando uma segunda etapa	26
9.4	Módulos	28
9.5	Containers	30
9.5.1	Use contêineres no Nextflow	31
10	Configuração	36
10.1	O arquivo <code>nextflow.config</code>	36
10.2	Movendo <code>params</code> para o config	36
10.3	Depois	36
10.4	Antes	37
10.5	Diretivas de processo	37
10.6	Tecnologia de empacotamento de software	38
10.7	Perfis (<code>profiles</code>)	38
10.8	Arquivo de parâmetros (<code>-params-file</code>)	39
10.9	Ordem de precedência	40
10.10	Inspecionar a config resolvida	40
10.11	Relatório de recursos	40
II	Hello nf-core	41
11	Hello nf-core	42
11.1	O que é nf-core	42
11.2	Configurar o diretório de trabalho	42
11.3	Recuperar o código do pipeline Demo	43
11.4	Examinar o perfil de teste	43
11.5	Examinar o workflow do pipeline	43

11.6 Executar o pipeline	43
12 Configurar a execução do pipeline	45
12.1 Parâmetros do pipeline	45
12.1.1 Listar todos os parâmetros com <code>--help</code>	45
12.1.2 Definir valores de parâmetros	46
12.2 Validação automática de parâmetros	46
12.3 Configuração propriamente dita	47
12.4 Personalizar recursos e argumentos com <code>-c</code>	47
12.5 <code>ext.args</code> — o padrão-chave para customização	48
13 Validação de entradas	49
13.1 Dois níveis de validação	49
13.2 Validação de parâmetros	49
13.3 Validação de <code>samplesheet</code>	50
13.4 Demonstração — <code>samplesheet</code> quebrada	50
13.5 Quando algo falha na validação	51
14 Usando módulos <code>nf-core</code>	52
14.1 Buscar módulos disponíveis	52
14.2 Instalar um módulo num pipeline	52
14.3 Estrutura do módulo instalado	53
14.4 Importar e usar	53
14.5 Atualizar e desinstalar	53
14.6 Personalização — o padrão <code>ext.args</code> de novo	53

1 Treinamento Nextflow

1.1 Visão geral do curso

Este curso é dividido em duas partes, no primeiro momento veremos como o nextflow é estruturado e no segundo momento aprenderemos a executar e personalizar pipelines já existentes. Cada sessão possui objetivos estruturados para introduzir informações gradualmente.

2 Ambiente de trabalho

2.1 Ambiente Local

Caso todas as dependências necessárias para execução desse minicurso estejam instaladas em sua máquina local, clone o repositório deste minicurso:

```
git clone git@github.com:dalmolingroup/nextflow_training.git  
  
cd nextflow_training
```

2.2 Ambiente Remoto

Caso não as dependências necessárias para este minicurso não estejam instaladas na sua máquina local, você poderá utilizar o ambiente pré-construído fornecido pela Seqera no [GitHub Codespace](#). Este ambiente de treinamento contém todo o software, código e dados necessários para trabalhar no curso de treinamento, então você não precisa instalar nada por conta própria.

3 Objetivos

Ao final do minicurso, você deverá ser capaz de:

- Entender a estrutura básica de um pipeline Nextflow (processos, canais, workflow, módulos)
- Executar pipelines nf-core prontos usando perfis de teste
- Configurar a execução via `params`, `-params-file` e arquivos de configuração
- Personalizar recursos e argumentos de ferramentas usando `ext.args`
- Instalar e usar módulos do repositório nf-core
- Diagnosticar erros comuns a partir das mensagens de validação de schema/samplesheet

3.1 O que este curso não cobre

- Programação em Nextflow do zero (não vamos escrever pipelines novos)
- Operadores avançados de canais (`flatten`, `groupTuple`, `join`, etc.)
- Desenvolvimento de módulos nf-core seguindo o template completo
- Criação de subworkflows

4 Pré-requisitos:

- Confortável com o terminal Linux (`cd`, `ls`, editar arquivos)
- Docker instalado e funcional (ou usar o ambiente remoto via GitHub Codespaces)
- Nextflow instalado (ou usar o ambiente remoto)
- `nf-core tools` (para a parte de módulos)

5 Agenda (5h)

Bloco	Duração	Conteúdo
1. Hello Nextflow — parte 1	45 min	História, estrutura de pipeline, primeiro <code>hello-world.nf</code> , <code>.command.*</code> , <code>-resume</code> , <code>nextflow log</code>
2. Hello Nextflow — parte 2	45 min	Canais, workflow multi-etapa, módulos, contêineres
Intervalo	15 min	
3. Configuração Nextflow	45 min	<code>nextflow.config</code> , diretivas de processo, <code>profiles</code> , <code>-params-file</code> , precedência
4. Hello nf-core — parte 1	60 min	O que é nf-core, <code>nextflow pull</code> , perfil <code>test</code> , <code>--help</code> , <code>-params-file</code> , <code>-c</code> , <code>ext.args</code>
Intervalo	15 min	
5. Validação de entrada	30 min	Schema de parâmetros, <code>schema</code> de <code>samplesheet</code> , debug de erros
6. Usando módulos nf-core	30 min	Catálogo, <code>nf-core modules install</code> , <code>ext.args</code> para customização
7. Q&A + prática livre	15 min	

Parte I

Hello Nextflow

6 Hello Nextflow

6.1 O que é Nextflow

Nextflow é uma linguagem de fluxo de trabalho (workflow language) criada em 2013 pelo Paolo Di Tommaso no Centre for Genomic Regulation (Barcelona), pensada para orquestrar pipelines científicos que envolvem muitas ferramentas e muitos dados.

Hoje é mantido pela **Seqera Labs** (a empresa fundada por Paolo) junto com uma comunidade global de contribuidores. A Seqera desenvolve o Nextflow como open source e oferece produtos comerciais em cima dele (Seqera Platform, Wave, Fusion), mas o Nextflow em si é livre.

6.2 O ecossistema nf-core

nf-core é uma comunidade independente que constrói pipelines Nextflow curados, revisados por pares e testados — hoje já são mais de 100 pipelines cobrindo genômica, transcriptômica, proteômica, metagenômica etc. Todos seguem o mesmo template, o que significa que aprender a rodar/customizar **um** pipeline nf-core te ensina a rodar **todos**.

6.3 Por que Nextflow

- **Reprodutibilidade:** cada processo roda em um contêiner (Docker/Singularity/Conda). Você não precisa instalar dependências manualmente.
- **Portabilidade:** o mesmo pipeline roda no seu laptop, num HPC (Slurm/PBS/SGE) ou na nuvem (AWS/Google/Azure), mudando só um perfil de configuração.
- **-resume:** se um passo falha ou você muda um parâmetro, o Nextflow reaproveita tudo que já rodou. Fundamental para pipelines longos.
- **Paralelismo automático:** você declara o fluxo de dados, o Nextflow paraleliza sozinho.
- **Comunidade:** com nf-core você raramente precisa escrever um pipeline do zero.

6.4 Defina o diretório de trabalho

No primeiro momento este curso, trabalharemos no diretório `hello-nextflow/`.

Mude de diretório executando este comando no terminal:

```
cd hello-nextflow/
```

Você pode configurar o VSCode para focar neste diretório, de modo que apenas os arquivos relevantes apareçam na barra lateral do explorador de arquivos:

```
code .
```

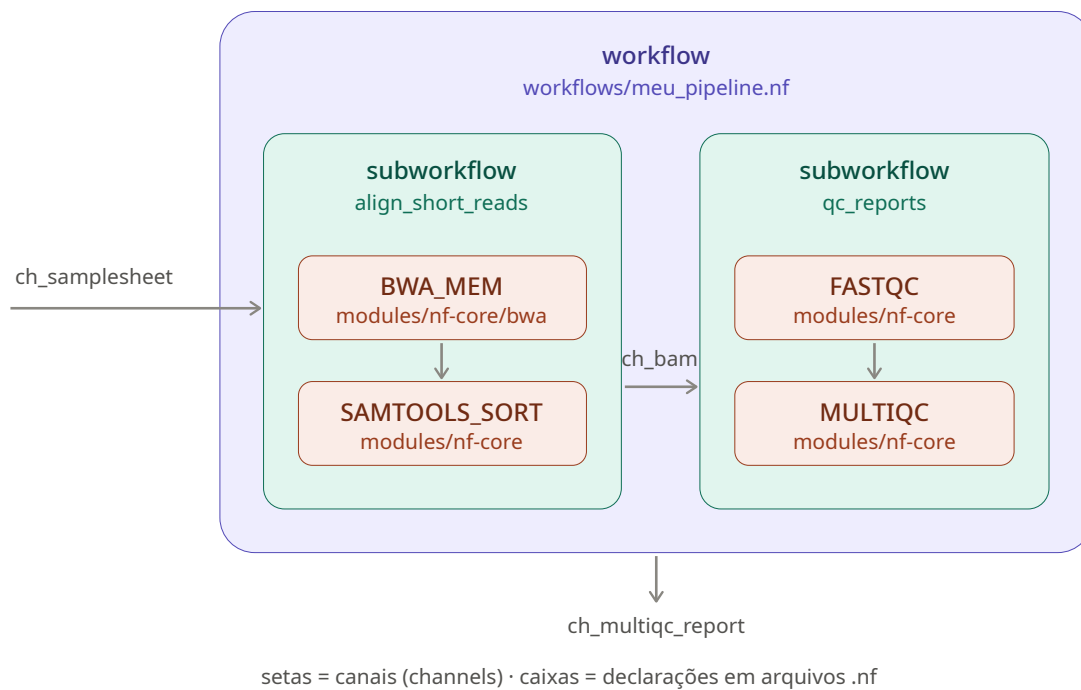
Explore os materiais fornecidos:

```
tree . -L 2
```

7 Como um pipeline em Nextflow é estruturado?

7.1 Estrutura de um pipeline

- **module:** É um processo atômico.
- **workflow:** Orquestra subworkflows e módulos. É o que define a ordem de execução.
- **channel:** É a aresta do grafo que carrega a informação para cada módulo/subworkflow.



7.2 Examinando um script nextflow

Fornecemos um script de workflow funcional, embora minimalista, chamado [hello-world.nf](#) que escreve 'Hello World!' com Nextflow.

Para começar, vamos abrir o script do fluxo de trabalho para que você tenha uma noção de como ele está estruturado. Então vamos executá-lo e procurar suas saídas.

 Ver conteúdo de `hello-world.nf`

```
#!/usr/bin/env nextflow

/*
 * Use echo to print 'Hello World!' to a file
 */
process sayHello {

    output:
    path 'output.txt'

    script:
    """
    echo 'Hello World!' > output.txt
    """
}

workflow {

    main:
    // emit a greeting
    sayHello()
}
```

Dica

Um script de fluxo de trabalho Nextflow normalmente inclui uma ou mais definições de process e o workflow em si, além de alguns blocos opcionais (não presentes aqui) que apresentaremos mais tarde.

7.2.1 Componentes do hello-world.nf

- **process:** O primeiro bloco de código descreve um **process**. O corpo do processo deve conter um bloco script que especifica o comando a ser executado, **que pode ser qualquer coisa que você seria capaz de executar em um terminal de linha de comando**.
 - Estrutura mínima
 - * Nome **sayHello**
 - * output neste caso inclui o qualificador **path**
 - * script o que será executado
- **workflow:** O segundo bloco de código descreve o workflow em si. A definição de fluxo de trabalho começa com a palavra-chave **workflow**, seguida por um nome opcional, depois o corpo do fluxo de trabalho delimitado por chaves.

7.2.2 Execute o workflow

Lance o fluxo de trabalho e monitore a execução, no terminal, execute o seguinte comando:

```
nextflow run hello-world.nf
```

7.2.3 Encontre a saída e os logs de execução

Quando você executa o Nextflow, é criado um diretório chamado **work** onde ele escreverá todos os arquivos gerados durante a execução.

Dentro do diretório **work**, para cada chamada de processo, o Nextflow cria um subdiretório aninhado, nomeado com um hash para torná-lo único, onde preparará todas as entradas necessárias, escreverá arquivos auxiliares e escreverá logs e quaisquer saídas do processo.

Para examinar o diretório **work** execute:

```
tree -a work
```

- **.command.begin:** Metadados relacionados ao início da execução da chamada do processo
- **.command.err:** Mensagens de erro (stderr) emitidas pela chamada do processo
- **.command.log:** Saída de log completa emitida pela chamada do processo
- **.command.out:** Saída regular (stdout) pela chamada do processo
- **.command.run:** Script completo executado pelo Nextflow para executar a chamada do processo
- **.command.sh:** O comando que foi realmente executado pela chamada do processo
- **.exitcode:** O código de saída resultante do comando

O arquivo `.command.sh` é especialmente útil porque informa o comando principal que o Nextflow executou, *não incluindo toda a contabilidade e configuração de tarefa/ambiente*.

Caso o workflow seja executado novamente um novo subdiretório com um conjunto completo de arquivos de saída e log será criado para cada execução. Isso mostra que executar o mesmo fluxo de trabalho várias vezes não sobrescreverá os resultados de execuções anteriores.

7.3 Aprimorando o Hello world

7.3.1 Publique outputs

A saída produzida pelo nosso pipeline está enterrada no diretório **work** várias camadas abaixo. Isso é feito propositalmente; o Nextflow está no controle desse diretório e não devemos interagir com ele. No entanto, isso torna inconveniente recuperar saídas que nos interessam.

Para contornar isso, podemos publicar os outputs em um diretório designado usando definições de saída em nível de workflow.

7.3.2 Uso básico

Isso vai envolver dois novos pedaços de código:

1. Um bloco **publish**: dentro do corpo do workflow, declarando saídas de processo.
2. Um bloco **output** no script especificando opções de saída como modo e localização.

Adicione o bloco **publish**:

7.4 Depois

```
workflow {  
  
    main:  
    // emite uma saudação  
    sayHello()  
  
    publish:  
    first_output = sayHello.out  
}
```


7.5 Antes

```
workflow {  
  
    main:  
    // emite uma saudação  
    sayHello()  
}
```

Nota

Você vê que podemos nos referir à saída do processo simplesmente fazendo `sayHello().out`, e atribuir a ela um nome arbitrário, `first_output`.

Adicione o bloco `output` ao script onde o caminho do diretório de saída será especificado:

7.6 Depois

```
workflow {  
  
    main:  
    // emite uma saudação  
    sayHello()  
  
    publish:  
    first_output = sayHello.out  
}  
  
output {  
    first_output {  
        path 'hello_world'  
    }  
}
```

7.7 Antes

```

workflow {

  main:
  // emite uma saudação
  sayHello()

  publish:
  first_output = sayHello.out
}

```

Execute o workflow modificado:

```
nextflow run hello-world.nf
```

7.8 Usando inputs variáveis

Em seu estado atual, nosso fluxo de trabalho usa uma saudação codificada no comando do processo. Queremos adicionar alguma flexibilidade usando uma variável de entrada, para que possamos mudar mais facilmente a saudação em tempo de execução.

Isso requer que façamos três conjuntos de mudanças em nosso script:

1. Alterar o processo para esperar uma entrada variável
2. Configurar um parâmetro de linha de comando para capturar a entrada do usuário
3. Passar a entrada para o processo no corpo do fluxo de trabalho

Vamos fazer essas mudanças uma de cada vez.

Primeiro, vamos adaptar a definição do processo para aceitar uma entrada chamada greeting.

7.9 Depois

```

process sayHello {

  input:
  val greeting

  output:
  path 'output.txt'
}

```

7.10 Antes

```
process sayHello {  
  
    output:  
    path 'output.txt'
```

Agora trocamos o valor codificado original pelo valor da variável de entrada que esperamos receber.

7.11 Depois

```
script:  
""  
echo '${greeting}' > output.txt  
""
```

7.12 Antes

```
script:  
""  
echo 'Hello World!' > output.txt  
""
```

Aqui, queremos criar um parâmetro chamado `--input`. Em princípio podemos escrevê-lo em qualquer lugar, mas podemos colocá-lo diretamente em `sayHello()`.

No bloco do fluxo de trabalho, faça a seguinte alteração de código:

7.13 Depois

```
// emite uma saudação  
sayHello(params.input)
```

7.14 Antes

```
// emite uma saudação  
sayHello()
```

Agora execute o workflow

```
nextflow run hello-world.nf --input 'Oi mundo!'
```

8 Gerenciando execução do workflow

Saber como lançar fluxos de trabalho e recuperar saídas é ótimo, mas você rapidamente descobrirá que há alguns outros aspectos do gerenciamento de fluxo de trabalho que tornarão sua vida mais fácil, especialmente se você estiver desenvolvendo seus próprios fluxos de trabalho.

Aqui mostramos como usar o recurso `-resume` para quando você precisar relançar o mesmo fluxo de trabalho, como inspecionar o log de execuções passadas com `nextflow log`, e como excluir diretórios work mais antigos com `nextflow clean`.

8.0.1 Relance um fluxo de trabalho com `-resume`

Às vezes, você vai querer executar novamente um pipeline que já lançou anteriormente sem refazer nenhuma etapa que já foi concluída com sucesso.

O Nextflow tem uma opção chamada `-resume` que permite fazer isso. Especificamente, neste modo, quaisquer processos que já foram executados com exatamente o mesmo código, configurações e entradas serão ignorados. Isso significa que o Nextflow só executará processos que você adicionou ou modificou desde a última execução, ou aos quais você está fornecendo novas configurações ou entradas.

Para usá-lo, basta adicionar `-resume` ao seu comando e executá-lo:

```
nextflow run hello-world.nf --input 'Konnichiwa!' -resume
```

8.0.2 Inspeção o log ou limpe execuções passadas

Sempre que você lança um fluxo de trabalho nextflow, uma linha é escrita em um arquivo de log chamado `history`, em um diretório oculto chamado `.nextflow` no diretório de trabalho atual.

i Ver exemplo do arquivo `.nextflow/history`

```
2024-11-05 10:32:15 5m 23s golden_cantor OK a1b2c3d4 1a2b3c4d-...
↪ nextflow run hello-world.nf
2024-11-05 10:41:07 3m 12s elegant_curie OK a1b2c3d4 5e6f7a8b-...
↪ nextflow run hello-world.nf --input 'Bonjour!'
2024-11-05 11:02:44 4m 58s naughty_lorenz ERR a1b2c3d4 9c0d1e2f-...
↪ nextflow run hello-world.nf --input 'Konnichiwa!' -resume
```

Cada linha: timestamp, duração, nome da execução, status, ID de revisão, ID de sessão, linha de comando.

Este arquivo fornece o timestamp, nome da execução, status, ID de revisão, ID de sessão e linha de comando completa para cada execução do Nextflow que foi lançada a partir do diretório de trabalho atual.

Uma maneira mais conveniente de acessar essas informações é usar o comando `nextflow log`.

```
nextflow log
```

Durante o processo de desenvolvimento, você normalmente executará seu rascunho de pipeline um grande número de vezes, o que pode levar a um acúmulo de muitos arquivos em muitos subdiretórios.

Felizmente, o Nextflow inclui um subcomando útil `clean` que pode excluir automaticamente os subdiretórios `work` de execuções passadas que você não se importa mais.

Existem várias opções para determinar o que excluir.

Aqui mostramos um exemplo que exclui todos os subdiretórios de execuções antes de uma determinada execução, especificada usando seu nome de execução.

Primeiro usamos a flag de execução de teste `-n` para verificar o que será excluído dado o comando:

```
nextflow clean -before golden_cantor -n
```

Se a saída parecer como esperado e você quiser prosseguir com a exclusão, execute novamente o comando com a flag `-f` em vez de `-n`:

```
nextflow clean -before golden_cantor -f
```

9 Componentes de um Workflow

9.1 Channels

Na maioria dos pipelines realistas, queremos processar múltiplos valores (vários arquivos FASTQ, por exemplo), então precisamos de uma maneira mais sofisticada de lidar com entradas.

É para isso que servem os canais do Nextflow. Canais são filas projetadas para lidar com entradas eficientemente e transportá-las de uma etapa para outra em fluxos de trabalho de múltiplas etapas, ao mesmo tempo que fornecem paralelismo integrado e muitos benefícios adicionais.

9.1.1 Forneça entradas variáveis através de um canal explicitamente

Existem várias fábricas de canais que podemos usar para configurar um canal. Para manter as coisas simples por enquanto, vamos usar a fábrica de canais mais básica, chamada `channel.of`, que criará um canal contendo um único valor. Funcionalmente, isso será semelhante a como tínhamos configurado antes, mas em vez de ter o Nextflow criando um canal implicitamente, estamos fazendo isso explicitamente agora.

Adicione o channel `greeting_ch` em [hello_channels.nf](#):

```
workflow {  
  
    main:  
    // cria um canal para entradas  
    greeting_ch = channel.of('Hello', 'Bonjour', 'Hola', "Oi")  
    // emite uma saudação  
    sayHello(greeting_ch)  
  
    publish:  
    first_output = sayHello.out  
}
```

Isso cria um canal chamado `greeting_ch` usando a fábrica de canais `channel.of()`, que configura um canal de fila simples, e carrega a string 'Hello Channels!' para usar como o valor da saudação.

Execute o workflow:

```
nextflow run hello-channels.nf
```

Os canais do Nextflow são construídos de uma maneira que nos permite operar em seu conteúdo usando operadores, que cobriremos em detalhes mais tarde neste capítulo.

Por enquanto, vamos apenas mostrar como usar um operador super simples chamado `view()` para inspecionar o conteúdo de um canal. Você pode pensar em `view()` como uma ferramenta de depuração, como uma instrução `print()` em Python, ou seu equivalente em outras linguagens.

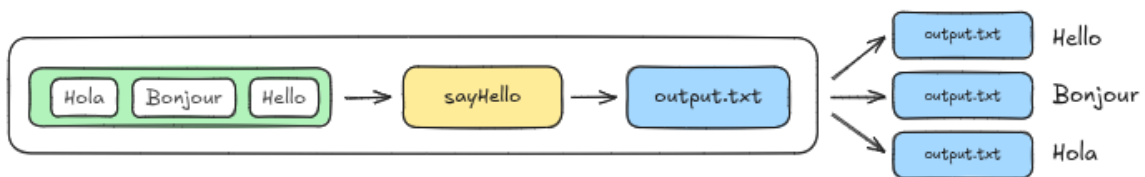
Adicione esta pequena linha ao bloco workflow:

```
workflow {  
  
    main:  
    // cria um canal para entradas  
    greeting_ch = channel.of('Hello Channels!')  
                                .view()  
  
    // emite uma saudação  
    sayHello(greeting_ch)  
  
    publish:  
    first_output = sayHello.out  
}
```

Agora execute o fluxo de trabalho novamente:

```
nextflow run hello-channels.nf
```

Como você pode ver, isso exibe o conteúdo do canal no console. Aqui temos apenas um elemento, mas quando começarmos a carregar múltiplos valores no canal na próxima seção, você verá que isso está configurado para exibir um elemento por linha.



9.2 Workflows

A maioria dos fluxos de trabalho do mundo real envolve mais de uma etapa. Neste módulo de treinamento, você aprenderá como conectar processos em um fluxo de trabalho com múltiplas etapas.

Agora veremos como realizar o seguinte:

1. Fazer os dados fluírem de um processo para o próximo
2. Coletar saídas de múltiplas chamadas de processo em uma única chamada de processo
3. Passar parâmetros adicionais para um processo
4. Lidar com múltiplas saídas provenientes de um processo

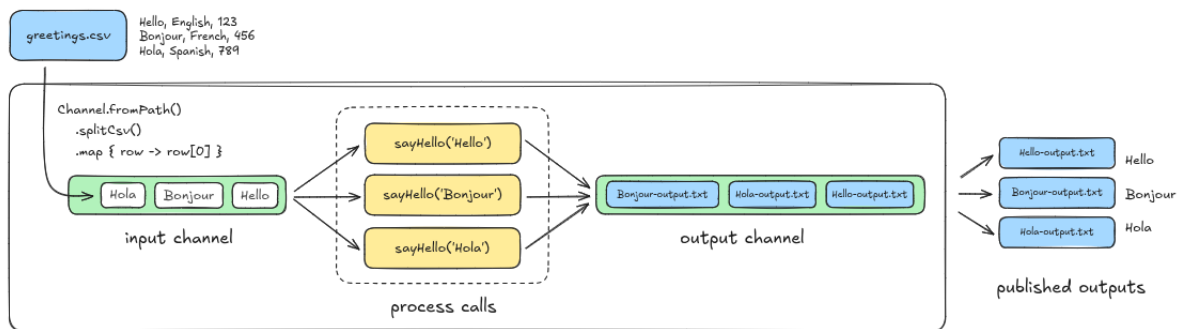
Para demonstrar, continuaremos construindo sobre o exemplo Hello World agnóstico de domínio das Partes 1 e 2. Desta vez, faremos as seguintes alterações em nosso fluxo de trabalho para refletir melhor como as pessoas constroem fluxos de trabalho reais:

1. Adicionar uma segunda etapa que converte a saudação para maiúsculas.
2. Adicionar uma terceira etapa que coleta todas as saudações transformadas e as escreve em um único arquivo.
3. Adicionar um parâmetro para nomear o arquivo de saída final e passá-lo como uma entrada secundária para a etapa de coleta.
4. Fazer a etapa de coleta também reportar uma estatística simples sobre o que foi processado.

Execute [hello-workflow.nf](#) :

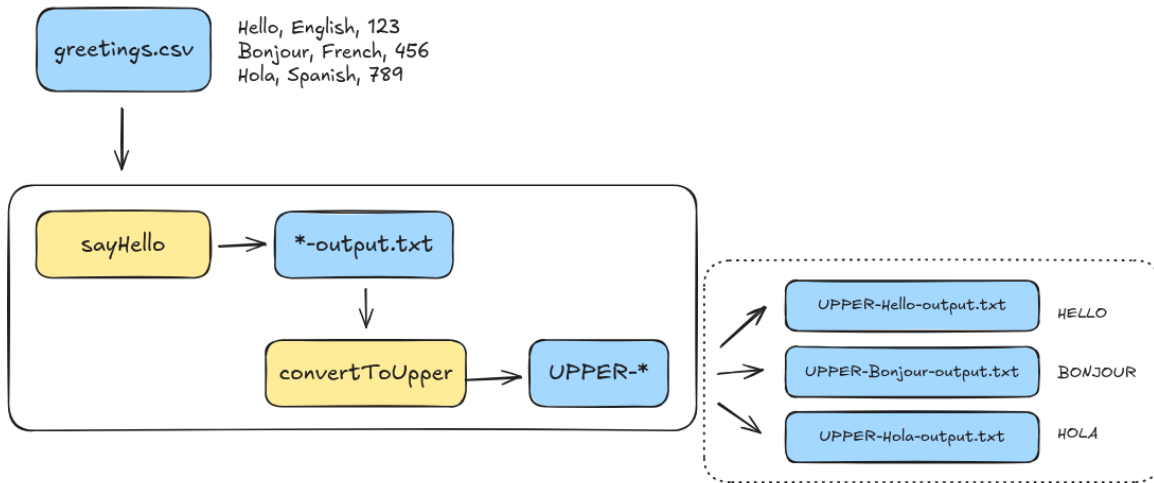
```
nextflow run hello-workflow.nf
```

Este diagrama resume a operação atual do fluxo de trabalho. Deve parecer familiar, exceto que agora estamos mostrando explicitamente que as saídas do processo são empacotadas em um canal, assim como as entradas foram. Vamos colocar esse canal de saída em bom uso em um minuto.



9.3 Adicionando uma segunda etapa

Vamos adicionar uma etapa para converter cada saudação para maiúsculas.



Para isso, precisamos fazer três coisas:

- Definir o comando que vamos usar para fazer a conversão para maiúsculas.
- Escrever um novo processo que envolve o comando de conversão para maiúsculas.
- Chamar o novo processo no bloco workflow e configurá-lo para receber a saída do processo sayHello() como entrada.

1. Comando de conversão para maiúsculas e teste-o no terminal:

```
tr '[a-z]' '[A-Z]'
```

2. Adicionando um novo processo em nosso workflow que irá executar esse comando

```
/*
 * Use uma ferramenta de substituição de texto para converter a saudação para
 * ↪ maiúsculas
 */
process convertToUpper {

    input:
    path input_file

    output:
    path "UPPER-${input_file}"
```

```

script:
  """
  cat '${input_file}' | tr '[a-z]' '[A-Z]' > 'UPPER-${input_file}'
  """
}

```

💡 Dica

Neste, componhamos o segundo nome de arquivo de saída com base no nome do arquivo de entrada, de forma similar ao que fizemos originalmente para a saída do primeiro processo.

3. Adicione uma chamada ao novo processo no bloco workflow

```

workflow {

  main:
    // cria um canal para entradas a partir de um arquivo CSV
    greeting_ch = channel.fromPath(params.input)
                  .splitCsv()
                  .map { line -> line[0] }

    // emite uma saudação
    sayHello(greeting_ch)
    // converte a saudação para maiúsculas
    convertToUpper()

  publish:
    first_output = sayHello.out
}

```

4. Passe a saída do primeiro processo para o segundo processo

```

// converte a saudação para maiúsculas
convertToUpper(sayHello.out)

```

5. Configure a publicação de saída do fluxo de trabalho

```

publish:
  first_output = sayHello.out
  uppercased = convertToUpper.out
}

```

```
output {
  first_output {
    path 'hello_workflow'
    mode 'copy'
  }
  uppercased {
    path 'hello_workflow'
    mode 'copy'
  }
}
```

Execute o fluxo de trabalho com `-resume`

```
nextflow run hello-workflow.nf -resume
```

9.4 Módulos

É uma boa prática armazenar seus módulos em um diretório específico. Você pode chamar esse diretório de qualquer nome, mas a convenção é chamá-lo de `modules/`.

```
mkdir modules
```

Na sua forma mais simples, transformar um processo existente em um módulo é pouco mais do que uma operação de copiar e colar. Vamos criar um esboço de arquivo para o módulo, copiar o código relevante e então excluí-lo do arquivo principal do fluxo de trabalho.

Então tudo o que precisaremos fazer é adicionar uma declaração `include` para que o Nextflow saiba trazer o código relevante em tempo de execução.

Crie um novo arquivo chamado `sayHello.nf` dentro da pasta `modules` e cole o processo `sayHello`

```
modules/sayHello.nf
```

```
/*
 * Usa echo para imprimir 'Hello World!' em um arquivo
 */
process sayHello {

  input:
  val greeting
```

```

    output:
    path "${greeting}-output.txt"

    script:
    """
    echo '${greeting}' > '${greeting}-output.txt'
    """
}

```

Uma vez feito isso, apague o processo sayHello do arquivo de fluxo de trabalho.

A sintaxe para incluir um processo de um módulo é bastante direta:

```
include { <PROCESS_NAME> } from '<path_to_module>'
```

Vamos inserir isso acima do bloco params e preenchê-lo adequadamente.

```

// Inclui módulos
include { sayHello } from './modules/sayHello.nf'

/*
 * Pipeline parameters
 */
params {
    input: Path = 'data/greetings.csv'
    batch: String = 'batch'
}

```

Execute o fluxo de trabalho

```
nextflow run hello-modules.nf -resume
```

Nota

Tentem modularizar o processo convertToUpper()

9.5 Containers

Nas Partes 1-4 deste curso de treinamento, você aprendeu a usar os blocos de construção básicos do Nextflow para montar um fluxo de trabalho simples capaz de processar texto, paralelizar a execução se houvesse múltiplas entradas e coletar os resultados para processamento adicional.

No entanto, você estava limitado às ferramentas básicas do UNIX disponíveis no seu ambiente. Tarefas do mundo real frequentemente requerem várias ferramentas e pacotes não incluídos por padrão. Normalmente, você precisaria instalar essas ferramentas, gerenciar suas dependências e resolver quaisquer conflitos.

Tudo isso é muito tedioso e irritante, então vamos mostrar como usar contêineres para resolver esse problema de forma muito mais conveniente.

Um contêiner é uma unidade de software leve, autônoma e executável criada a partir de uma imagem de contêiner que inclui tudo o que é necessário para executar uma aplicação, incluindo código, bibliotecas do sistema e configurações. Como você pode imaginar, isso será muito útil para tornar seus pipelines mais reproduzíveis.

Execute `hello-containers.nf`

```
nextflow run hello-containers.nf
```

Vamos usar um “container” manualmente

Baixe a imagem do container

```
docker pull 'community.wave.sequera.io/library/cowpy:1.1.5--3db457ae1977a273'
```

Use o contêiner para executar `cowpy` como um comando único

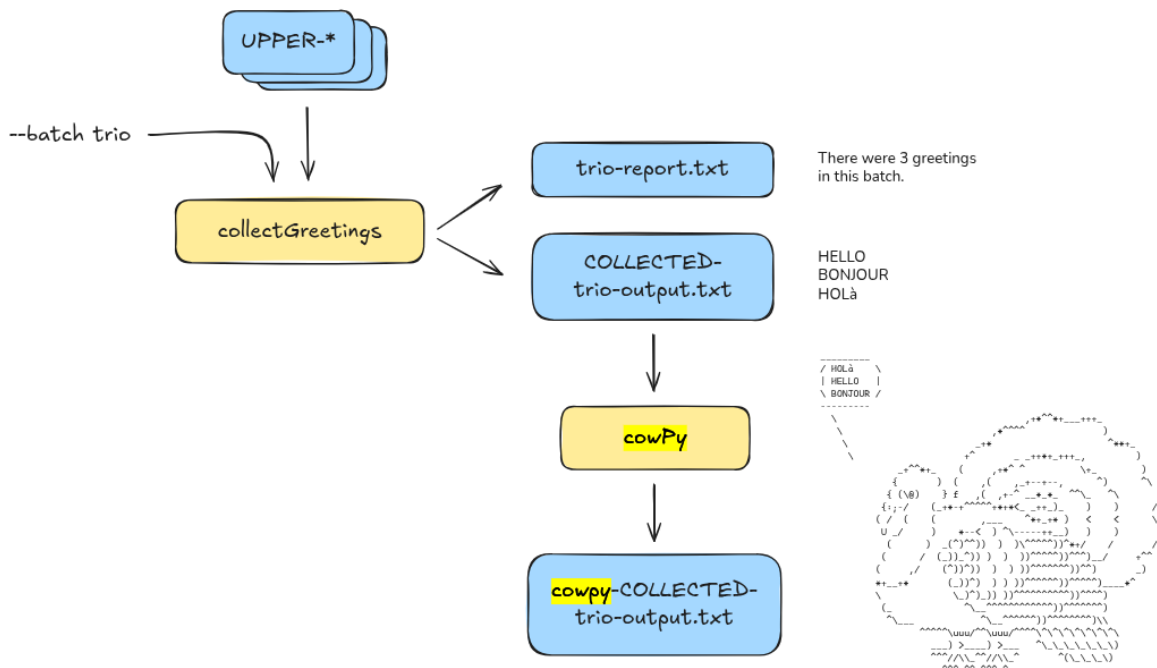
```
docker run --rm -it
↪ 'community.wave.sequera.io/library/cowpy:1.1.5--3db457ae1977a273'
↪ /bin/bash
```

Execute o(s) comando(s) da ferramenta desejada `yirdtk echo "Hello Containers" | cowpy -c tux`

9.5.1 Use contêineres no Nextflow

O Nextflow tem suporte integrado para executar processos dentro de contêineres para permitir que você execute ferramentas que não tem instaladas no seu ambiente de computação. Isso significa que você pode usar qualquer imagem de contêiner que desejar para executar seus processos, e o Nextflow cuidará de puxar a imagem, montar os dados e executar o processo dentro dela.

Para demonstrar isso, vamos adicionar uma etapa cowpy ao pipeline que estivemos desenvolvendo, após a etapa collectGreetings.



i Nota

Tentem escrever o módulo cowpy.nf

💡 Mostrar resposta

Podemos modelar nosso processo cowpy nos outros processos que escrevemos anteriormente.

```
#!/usr/bin/env nextflow

// Gera arte ASCII com cowpy
process cowpy {

    input:
    path input_file
    val character

    output:
    path "cowpy-${input_file}"

    script:
    """
    cat ${input_file} | cowpy -c "${character}" > cowpy-${input_file}
    """
}
```

O processo espera um `input_file` contendo as saudações, bem como um valor `character`. A saída será um novo arquivo de texto contendo a arte ASCII gerada pela ferramenta `cowpy`.

Adicione `cowpy` ao fluxo de trabalho

```
// Inclui módulos
include { sayHello } from './modules/sayHello.nf'
include { convertToUpper } from './modules/convertToUpper.nf'
include { collectGreetings } from './modules/collectGreetings.nf'
include { cowpy } from './modules/cowpy.nf'
```

Vamos conectar o processo `cowpy()` à saída do processo `collectGreetings()`, que como você deve se lembrar produz duas saídas:

- `collectGreetings.out.outfile` contém o arquivo de saída *<-o que queremos*
- `collectGreetings.out.report` contém o arquivo de relatório com a contagem de saudações por lote


```

main:
// cria um canal para entradas de um arquivo CSV
greeting_ch = channel.fromPath(params.input)
                .splitCsv()
                .map { line -> line[0] }

// emite uma saudação
sayHello(greeting_ch)
// converte a saudação para maiúsculas
convertToUpper(sayHello.out)
// coleta todas as saudações em um arquivo
collectGreetings(convertToUpper.out.collect(), params.batch)
// gera arte ASCII das saudações com cowpy
cowpy(collectGreetings.out.outfile, params.character)

```

Adicione o parâmetro character ao bloco param

```

/*
 * Parâmetros do pipeline
 */
params {
    input: Path = 'data/greetings.csv'
    batch: String = 'batch'
    character: String = 'turkey'
}

```

Atualize as saídas do workflow

```

publish:
first_output = sayHello.out
uppercased = convertToUpper.out
collected = collectGreetings.out.outfile
batch_report = collectGreetings.out.report
cowpy_art = cowpy.out

```

Atualize o bloco output

```

output {
  first_output {
    path 'hello_containers/intermediates'
    mode 'copy'
  }
  uppercased {
    path 'hello_containers/intermediates'
    mode 'copy'
  }
  collected {
    path 'hello_containers/intermediates'
    mode 'copy'
  }
  batch_report {
    path 'hello_containers'
    mode 'copy'
  }
  cowpy_art {
    path 'hello_containers'
    mode 'copy'
  }
}

```

Execute o workflow

```
nextflow run hello-containers.nf
```

None

Oh não, há um erro! O código de erro dado por error exit status (127) significa que o executável que pedimos não foi encontrado.

Isso faz sentido, já que estamos chamando a ferramenta cowpy mas ainda não especificamos um contêiner (ops).

Use um contêiner para executar o processo cowpy

```

process cowpy {
  container
  ↪ 'community.wave.seqera.io/library/cowpy:1.1.5--3db457ae1977a273'

  input:
  path input_file
  val character

  output:
  path "cowpy-${input_file}"

  script:
  """
  cat ${input_file} | cowpy -c "${character}" > cowpy-${input_file}
  """
}

```

Execute o workflow

```
nextflow run hello-containers.nf
```

10 Configuração

Todo pipeline sério é executado em contextos diferentes: laptop pessoal, cluster HPC, nuvem. O objetivo do arquivo `nextflow.config` é permitir mudar **como** o pipeline roda **sem tocar** no código do workflow.

10.1 O arquivo `nextflow.config`

Quando você roda `nextflow run <script>`, o Nextflow carrega automaticamente um arquivo chamado `nextflow.config` no diretório atual (se existir).

Ele pode conter:

- Valores padrão de parâmetros (`params`)
- Diretivas de processo (`cpus`, `memory`, `container`, `publishDir`, ...)
- Habilitação de contêineres/conda (`docker.enabled`, `conda.enabled`, ...)
- Perfis de configuração (`profiles { ... }`)
- Configurações de saída (`outputDir`, `workflow.output.mode`)

10.2 Movendo `params` para o config

Manter valores padrão no arquivo de configuração deixa o código do workflow mais limpo e permite alterar valores sem editar o script.

10.3 Depois

`nextflow.config`:

```
docker.enabled = true

params {
    input      = 'data/greetings.csv'
    batch      = 'batch'
```

```
    character = 'turkey'
}
```

No workflow, mantemos só a declaração dos tipos:

```
params {
    input: Path
    batch: String
    character: String
}
```

10.4 Antes

nextflow.config:

```
docker.enabled = true
```

No workflow, os valores padrão estavam no próprio script:

```
params {
    input: Path      = 'data/greetings.csv'
    batch: String    = 'batch'
    character: String = 'turkey'
}
```

10.5 Diretivas de processo

As diretivas controlam recursos e comportamento de execução dos processos.

```
process {
    cpus    = 1
    memory = 1.GB

    withName: 'cowpy' {
        cpus    = 2
        memory = 2.GB
    }
}
```

- Bloco `process { ... }`: aplica a **todos** os processos
- `withName: 'nome'`: aplica só a um processo específico
- Diretivas comuns: `cpus`, `memory`, `time`, `container`, `conda`, `publishDir`, `errorStrategy`

10.6 Tecnologia de empacotamento de software

O mesmo pipeline pode rodar com Docker, Singularity ou Conda — basta habilitar no config.

```
docker.enabled      = true
// ou
singularity.enabled = true
// ou
conda.enabled       = true
```

E no módulo, cada processo declara sua imagem/pacote:

```
process cowpy {
  container
  ↪ 'community.wave.sequera.io/library/cowpy:1.1.5--3db457ae1977a273'
  conda      'conda-forge::cowpy==1.1.5'
  ...
}
```

Dica

Se Docker e Conda estiverem habilitados juntos, o Nextflow **prioriza contêineres**.

10.7 Perfis (profiles)

Perfis são conjuntos nomeados de configuração que você ativa em tempo de execução com `-profile`.

```
profiles {
  my_laptop {
    process.executor = 'local'
    docker.enabled   = true
  }
  hpc {
```

```

    process.executor = 'slurm'
    conda.enabled     = true
    process.resourceLimits = [
        memory: 750.GB,
        cpus:   200,
        time:   30.d
    ]
}
test {
    params.input      = 'data/greetings.csv'
    params.batch      = 'test'
    params.character  = 'dragonandcow'
}
}

```

Ativação:

```

nextflow run hello-config.nf -profile my_laptop
nextflow run hello-config.nf -profile my_laptop,test # combinando perfis

```

i Nota

Este é exatamente o mecanismo que pipelines nf-core usam com `-profile docker,test` — que veremos na próxima parte do curso.

10.8 Arquivo de parâmetros (`-params-file`)

Quando você tem muitos parâmetros ou quer distribuir uma configuração de execução, um arquivo YAML/JSON é mais prático que a linha de comando.

`test-params.yaml`:

```

input:      "data/greetings.csv"
batch:      "yaml"
character:  "stegosaurus"

```

Execução:

```
nextflow run hello-config.nf -params-file test-params.yaml
```

! Importante

No arquivo de parâmetros use : (YAML). No `nextflow.config` use = (Groovy).

10.9 Ordem de precedência

Quando o mesmo parâmetro é definido em vários lugares, o Nextflow segue esta ordem (do mais forte ao mais fraco):

1. Linha de comando (`--param valor`)
2. Arquivo de parâmetros (`-params-file`)
3. Arquivos de configuração (`-c custom.config`, depois `nextflow.config`)
4. Valores padrão no bloco `params` do workflow

10.10 Inspecionar a config resolvida

Para descobrir com quais valores o pipeline vai rodar **sem executar nada**:

```
nextflow config
nextflow config -profile my_laptop,test
```

O comando devolve toda a configuração já resolvida — muito útil para debug quando algo não está pegando o valor esperado.

10.11 Relatório de recursos

Para descobrir quanto CPU/memória cada processo realmente usa:

```
nextflow run hello-containers.nf -with-report relatorio.html
```

Isso gera um HTML com utilização por processo — base para ajustar `cpus/memory` no config.

Parte II

Hello nf-core

11 Hello nf-core

11.1 O que é nf-core

nf-core é uma comunidade global que desenvolve e mantém pipelines Nextflow **curados, revisados por pares e prontos para produção**. Todos seguem o mesmo template, então uma vez que você aprende a estrutura, sabe navegar em qualquer pipeline da coleção.

O que a comunidade entrega:

- **Pipelines** (nf-co.re/pipelines) — mais de 100, cobrindo genômica, transcriptômica, proteômica, metagenômica, etc.
- **Módulos reutilizáveis** (nf-co.re/modules) — processos wrapping ferramentas comuns (FASTQC, samtools, GATK...) que qualquer pipeline pode instalar.
- **Configurações institucionais** (nf-co.re/configs) — perfis prontos para HPCs de dezenas de universidades.
- **Ferramenta CLI nf-core** — para criar, testar e instalar pipelines/módulos.

O ganho prático para quem faz bioinformática: você não escreve pipelines do zero, você **usa** e **customiza** o que a comunidade já validou.

11.2 Configurar o diretório de trabalho

Mude de diretório agora executando este comando no terminal:

```
cd hello_nfcore
```

Nesta primeira parte do curso de treinamento Hello nf-core, mostramos como encontrar e experimentar um pipeline do nf-core, configurar e personalizar sua execução para suas necessidades, e entender como a validação de entrada protege contra erros comuns.

Vamos usar um pipeline chamado [nf-core/demo](https://nf-co.re/demo) que é mantido pelo projeto nf-core como parte de seu inventário de pipelines para fins de demonstração e treinamento.

Certifique-se de que seu diretório de trabalho esteja definido como hello-nf-core/

11.3 Recuperar o código do pipeline Demo

Depois de determinarmos que o pipeline parece ser adequado para nossos propósitos, vamos experimentá-lo. Felizmente, o Nextflow facilita a recuperação de pipelines de repositórios formatados corretamente sem precisar baixar nada manualmente.

```
nextflow pull nf-core/demo
```

Você pode fazer isso com qualquer pipeline Nextflow que esteja configurado adequadamente no GitHub, não apenas pipelines do nf-core. No entanto, o nf-core é a maior coleção de código aberto de pipelines Nextflow.

Para facilitar a navegação pelo código-fonte do pipeline, crie um link simbólico apontando para a cópia do pipeline que foi baixada:

```
mkdir -p pipelines/nf-core
ln -s "$(echo $NXF_HOME/assets/.repos/nf-core/demo/clones/*/)"
↪ pipelines/nf-core/demo
```

11.4 Examinar o perfil de teste

É uma boa prática verificar o que o perfil de teste de um pipeline especifica antes de executá-lo. O perfil test para nf-core/demo está no arquivo de configuração conf/test.config. Você pode encontrá-lo localmente dentro do código-fonte do pipeline que o nextflow pull baixou.

```
code pipelines/nf-core/demo/conf/test.config
```

11.5 Examinar o workflow do pipeline

```
code pipelines/nf-core/demo/main.nf
```

11.6 Executar o pipeline

```
nextflow run nf-core/demo -profile docker,test --outdir demo-results
```

12 Configurar a execução do pipeline

O nf-core distingue dois níveis de configuração:

Nível	O que é	Como definir
Parâmetros do pipeline (params)	Entradas, flags de análise, comportamento de ferramentas	<code>--param</code> , <code>-params-file file.yml</code> , bloco <code>params</code> no <code>config</code>
Configuração propriamente dita	Como o pipeline é executado — executor, recursos, engine de contêiner	<code>nextflow.config</code> , <code>-c custom.config</code> , <code>-profile</code>

12.1 Parâmetros do pipeline

12.1.1 Listar todos os parâmetros com `--help`

Todo pipeline nf-core expõe seus parâmetros com uma flag `--help`, alimentada pelo `nextflow_schema.json`:

```
nextflow run nf-core/demo --help
```

A saída agrupa os parâmetros por categoria (Input/output, Reference genome, etc.) com tipo, descrição e valor padrão.

Dica

Use `nextflow run nf-core/demo --help --show_hidden` para ver parâmetros ocultos, como `--publish_dir_mode`.

12.1.2 Definir valores de parâmetros

Três formas equivalentes:

1. **Linha de comando** (rápido, um valor por vez):

```
nextflow run nf-core/demo -profile docker,test --outdir demo-results
↪ --skip_trim
```

2. **Arquivo de parâmetros com -params-file** (recomendado):

my_params.yml:

```
skip_trim: true
```

```
nextflow run nf-core/demo -profile docker,test --outdir demo-results
↪ -params-file my_params.yml
```

3. **Arquivo de configuração personalizado com -c** — funciona, mas para params prefira as opções 1 e 2 (precedência é mais previsível).

! Importante

Booleans na linha de comando: a partir do Nextflow 26.04, valores da CLI são tratados como strings. Passar `--skip_trim` puro pode falhar validação. Para booleans, use `-params-file`.

12.2 Validação automática de parâmetros

O nf-schema valida parâmetros contra o `nextflow_schema.json` **antes** de executar qualquer processo — falha rápida:

Parâmetro inexistente (só warning, mas alerta typos):

```
nextflow run nf-core/demo -profile docker,test --outdir demo --outDir demo
# WARN: --outDir: invalid
```

Valor de tipo errado (falha imediata):

```
nextflow run nf-core/demo -profile docker,test --outdir demo --skip_trim yes
# ERROR: --skip_trim (yes): Value is [string] but should be [boolean]
```

Isso economiza horas de execução perdidas por erros de digitação ou tipo.

12.3 Configuração propriamente dita

Os arquivos de config do pipeline ficam em `pipelines/nf-core/demo/conf/`. Os mais importantes:

- **base.config**: define labels de recurso (`process_low`, `process_medium`, `process_high`) — CPUs/memória/tempo por categoria de processo.
- **modules.config**: define `ext.args` (argumentos extras por ferramenta) e configurações de `publishDir` por processo.
- **test.config**: perfil de teste com dataset mínimo (ativado por `-profile test`).

Regra de ouro: nunca edite esses arquivos diretamente. Crie um `custom.config` e passe com `-c` — os valores dele sobrescrevem os padrões.

12.4 Personalizar recursos e argumentos com `-c`

`custom.config`:

```
process {
  withName: 'FASTQC' {
    cpus    = 2
    memory = 4.GB
  }
  withName: 'SEQTK_TRIM' {
    ext.args = '-b 5'
  }
}
```

- Primeiro bloco: reduz recursos alocados ao FASTQC.
- Segundo bloco: passa a flag extra `-b 5` ao `seqtk trimfq` (corta 5 bases do início de cada read).

Executar:

```
nextflow run nf-core/demo -profile docker,test --outdir demo-results-custom  
↪ -c custom.config
```

Para confirmar que a flag entrou, procure o hash de SEQTK_TRIM na saída e inspecione o comando:

```
cat work/17/428668*/.command.sh
```

12.5 ext.args — o padrão-chave para customização

Toda ferramenta CLI tem dezenas de flags. Os módulos nf-core não expõem todas como parâmetros; em vez disso, deixam um “gancho” chamado `ext.args` que é passado direto para a ferramenta.

Aviso

`ext.args` substitui o valor padrão do módulo, não adiciona. Se FASTQC já tem `ext.args = '--quiet'` em `conf/modules.config` e você define `ext.args = '--kmers 8'`, o `--quiet` desaparece. Para manter ambos: `ext.args = '--quiet --kmers 8'`.

Sempre inspecione `conf/modules.config` antes de sobrescrever:

```
grep -A5 "withName: FASTQC" pipelines/nf-core/demo/conf/modules.config
```


13 Validação de entradas

Sem validação, você pode esperar horas até um pipeline falhar num arquivo inexistente. O nf-core resolve isso com o plugin **nf-schema**, que valida **antes** de qualquer processo rodar.

13.1 Dois níveis de validação

Arquivo	O que valida
nextflow_schema.json	Parâmetros do pipeline (--input, --outdir, --skip_trim, ...)
assets/schema_input.json	Estrutura da samplesheet (colunas, tipos, arquivos existentes)

Ambos usam o padrão [JSON Schema](#).

i Nota

A validação de samplesheet checa a **estrutura** (colunas certas, arquivos existem, extensão correta), não o **conteúdo** dos arquivos (integridade do FASTQ, por exemplo).

13.2 Validação de parâmetros

O nf-core/demo exibe todos os parâmetros e suas restrições via --help. Cada campo do schema define tipo, formato e regras.

Exemplo de erro por tipo errado:

```
nextflow run nf-core/demo -profile docker,test --outdir demo --skip_trim yes
```

```
ERROR ~ Validation of pipeline parameters failed!  
* --skip_trim (yes): Value is [string] but should be [boolean]
```

O pipeline **para antes** de rodar qualquer processo.

13.3 Validação de samplesheet

O schema `assets/schema_input.json` define para cada coluna: nome, tipo, obrigatoriedade, padrão (regex), se o arquivo precisa existir e a mensagem de erro amigável.

Exemplo real do `nf-core/demo`:

```
{
  "type": "object",
  "properties": {
    "sample": {
      "type": "string",
      "pattern": "^\\S+$",
      "errorMessage": "Sample name must be provided and cannot contain
        ↪ spaces"
    },
    "fastq_1": {
      "type": "string",
      "format": "file-path",
      "exists": true,
      "pattern": "^([\\S\\s]*\\/?)?[\\s\\/?]+\\.f(ast)?q\\.gz$",
      "errorMessage": "FastQ file for reads 1 must be provided..."
    },
    "fastq_2": { "...": "..." }
  },
  "required": ["sample", "fastq_1"]
}
```

`fastq_1` é obrigatório; `fastq_2` é opcional (suporta single-end).

13.4 Demonstração — samplesheet quebrada

`malformed_samplesheet.csv`:

```
sample,fastq_2
SAMPLE1,/not/a/real/file.fastq.gz
```

Este arquivo tem dois problemas: falta a coluna `fastq_1` e o caminho em `fastq_2` não existe.

```
nextflow run nf-core/demo -profile docker,test --outdir demo --input  
↳ malformed_samplesheet.csv
```

ERROR ~ Validation of pipeline parameters failed!

```
* --input (malformed_samplesheet.csv): Validation of file failed:  
  -> Entry 1: Error for field 'fastq_2' (/not/a/real/file.fastq.gz): the  
      file  
        or directory does not exist  
  -> Entry 1: Missing required field(s): fastq_1
```

Dica

O nf-schema reporta **todos os erros de uma vez**, não só o primeiro. Você corrige tudo em uma iteração.

13.5 Quando algo falha na validação

Fluxo de debug:

1. Ler a mensagem — quase sempre diz exatamente qual campo/valor está errado.
2. Se for parâmetro: consultar `nextflow run <pipeline> --help` para ver tipo/formato esperado.
3. Se for samplesheet: abrir `assets/schema_input.json` do pipeline para ver o schema exato.
4. Corrigir e reexecutar — não precisa `-resume`, nada foi computado ainda.

14 Usando módulos nf-core

Todo pipeline nf-core é composto por **módulos** — processos que envolvem uma ferramenta específica (FASTQC, samtools, GATK...). Muitos são compartilhados entre pipelines através do repositório [nf-core/modules](https://nf-co.re/modules).

Isso significa que, ao customizar ou construir um pipeline, você raramente precisa escrever um processo do zero: instale o módulo pronto e use.

14.1 Buscar módulos disponíveis

Pelo site: nf-co.re/modules — busca com filtro e página de doc para cada módulo (inputs, outputs, exemplo de uso).

Pela linha de comando (requer `nf-core tools` instalado):

```
nf-core modules list remote          # lista todos
nf-core modules list remote | grep -i samtools  # filtra
nf-core modules info samtools/view        # detalhes de um módulo
```

Nota

Convenção de nomes: o / na URL vira _ em alguns contextos. `samtools/view` = pacote `samtools`, comando `view`. Módulos com um único comando principal têm nome curto: `fastqc`, `multiqc`.

14.2 Instalar um módulo num pipeline

Dentro do diretório raiz de um pipeline nf-core:

```
nf-core modules install samtools/view
```

Isso faz três coisas:

1. Baixa os arquivos do módulo para `modules/nf-core/samtools/view/`.
2. Registra o módulo em `modules.json` (com SHA para reprodutibilidade).
3. Devolve a linha de `include` pronta para colar no workflow.

14.3 Estrutura do módulo instalado

```
modules/nf-core/samtools/view/  
  main.nf          # o processo em si  
  meta.yml         # descrição, inputs, outputs  
  environment.yml  # dependências conda  
  tests/           # testes automatizados
```

14.4 Importar e usar

```
include { SAMTOOLS_VIEW } from '../modules/nf-core/samtools/view/main'
```

Convenção: nomes de módulos importados ficam em **MAIÚSCULAS**.

14.5 Atualizar e desinstalar

```
nf-core modules update samtools/view    # atualiza para versão mais recente  
nf-core modules uninstall samtools/view # remove
```

14.6 Personalização — o padrão `ext.args` de novo

Como todo módulo nf-core respeita `ext.args`, você quase nunca precisa **editar** o código do módulo. Basta passar os argumentos extras via `custom.config`:

```
process {  
  withName: 'SAMTOOLS_VIEW' {  
    ext.args = '-b -q 30 -f 2'  
  }  
}
```

Se precisar de algo que `ext.args` não cobre (mudar a lógica do processo, adicionar um output), aí sim vale editar. Mas antes, **cheque se o módulo aceita a customização por argumento** — quase sempre aceita.